

Chapter 29

SHA Accelerator (SHA)

29.1 Introduction

ESP32-P4 integrates an SHA accelerator, which is a hardware device that speeds up the SHA algorithm significantly, compared to an SHA algorithm implemented solely in software. The SHA accelerator integrated in ESP32-P4 has two working modes, which are [Typical SHA](#) and [DMA-SHA](#).

29.2 Features

The following functionality is supported:

- The following hash algorithms introduced in [FIPS PUB 180-4 Spec](#).
 - SHA-1
 - SHA-224
 - SHA-256
 - SHA-384
 - SHA-512
 - SHA-512/224
 - SHA-512/256
 - SHA-512/t
- Two working modes
 - Typical SHA
 - DMA-SHA
- Interleaved function when working in Typical SHA working mode
- Interrupt function when working in DMA-SHA working mode

29.3 Working Modes

The SHA accelerator integrated in ESP32-P4 has two working modes.

- [Typical SHA Working Mode](#): all the data is written and read via CPU directly.
- [DMA-SHA Working Mode](#): all the data is read via DMA. That is, users can configure the GDMA-AXI controller to read all the data needed for hash operation, thus releasing CPU for completing other tasks.

The SHA accelerator is activated by setting the [HP_SYS_CLKRST_CRYPT_SHA_CLK_EN](#) register field and clearing the [HP_SYS_CLKRST_RST_EN_SHA](#) register field. Additionally, due to the hardware reuse relationship between cryptography modules, set the related reset register field of [RSA Digital Signature Peripheral \(RSA_DS\)](#), [HMAC Accelerator \(HMAC\)](#), and [ECDSA Digital Signature Peripheral \(ECDSA_DS\)](#) to 1 can reset the SHA module. Therefore, users also need to clear [HP_SYS_CLKRST_RST_EN_DS](#), [HP_SYS_CLKRST_RST_EN_HMAC](#), and [HP_SYS_CLKRST_RST_EN_ECDSA](#) fields.

Users can start the SHA accelerator with different working modes by configuring registers [SHA_START_REG](#) and [SHA_DMA_START_REG](#). For details, please see Table 29.3-1.

Table 29.3-1. SHA Accelerator Working Mode

Working Mode	Configuration Method
Typical SHA	Set SHA_START_REG to 1
DMA-SHA	Set SHA_DMA_START_REG to 1

Users can choose hash algorithms by configuring the [SHA_MODE_REG](#) register. For details, please see Table 29.3-2.

Table 29.3-2. SHA Hash Algorithm Selection

Hash Algorithm	SHA_MODE_REG Configuration
SHA-1	0
SHA-224	1
SHA-256	2
SHA-384	3
SHA-512	4
SHA-512/224	5
SHA-512/256	6
SHA-512/t	7

Notice:

ESP32-P4's [RSA Digital Signature Peripheral \(RSA_DS\)](#) and [HMAC Accelerator \(HMAC\)](#) modules also call the SHA accelerator when working. Therefore, users cannot access the SHA accelerator when these modules are working.

29.4 Function Description

The SHA accelerator generates the message digest via two steps: [Preprocessing](#) and [Hash operation](#).

29.4.1 Preprocessing

Preprocessing consists of three steps: [padding the message](#), [parsing the message into message blocks](#) and [setting the initial hash value](#).

29.4.1.1 Padding the Message

The SHA accelerator can only process message blocks of 512 or 1024 bits, depending on the algorithm. Thus, all the messages should be padded to a multiple of 512 bits or 1024 bits before the hash operation.

Suppose that the length of the message M is m bits. Then M shall be padded as introduced below:

- **SHA-1, SHA-224 and SHA-256**
 1. First, append the bit “1” to the end of the message;
 2. Second, append k bits of zeros, where k is the smallest, non-negative solution to the equation $m + 1 + k \equiv 448 \bmod 512$;
 3. Last, append the 64-bit block of value equal to the number m expressed using a binary representation.
- **SHA-384, SHA-512, SHA-512/224, SHA-512/256 and SHA-512/t**
 1. First, append the bit “1” to the end of the message;
 2. Second, append k zero bits, where k is the smallest, non-negative solution to the equation $m + 1 + k \equiv 896 \bmod 1024$;
 3. Last, append the 128-bit block of value equal to the number m expressed using a binary representation.

For more details, please refer to [FIPS PUB 180-4 Spec](#) > Section “Padding the Message”.

29.4.1.2 Parsing the Message

The message and its padding must be parsed into N 512-bit or 1024-bit blocks. During the task, all the message blocks should be written into the [SHA_M_n_REG](#).

- For **SHA-1, SHA-224 and SHA-256**:

The message and its padding are parsed into N 512-bit blocks, $M^{(1)}, M^{(2)}, \dots, M^{(N)}$. Since the 512 bits of the input block may be expressed as sixteen 32-bit words, the first 32 bits of message block i are denoted $M_0^{(i)}$, the next 32 bits are $M_1^{(i)}$, and so on up to $M_{15}^{(i)}$.

During the task, $M_0^{(i)}$ should be stored in [SHA_M_O_REG](#), $M_1^{(i)}$ stored in [SHA_M_1_REG](#), ..., and $M_{15}^{(i)}$ stored in [SHA_M_15_REG](#).

- For **SHA-384, SHA-512, SHA-512/224, SHA-512/256 and SHA-512/t**:

The message and its padding are parsed into N 1024-bit blocks. Since the 1024 bits of the input block may be expressed as sixteen 64-bit words, the first 64 bits of message block i are denoted $M_0^{(i)}$, the next 64 bits are $M_1^{(i)}$, and so on up to $M_{15}^{(i)}$.

During the task, the most significant 32 bits and the least significant 32 bits of $M_0^{(i)}$ should be stored in [SHA_M_O_REG](#) and [SHA_M_1_REG](#), respectively, ..., the most significant 32 bits and the least significant 32 bits of $M_{15}^{(i)}$ should be stored in [SHA_M_30_REG](#) and [SHA_M_31_REG](#), respectively.

Note:

For more information about “message block”, please refer to [FIPS PUB 180-4 Spec](#) > Section “Glossary of Terms and Acronyms”.

29.4.1.3 Setting the Initial Hash Value

Before hash task begins for each of the secure hash algorithms, the initial Hash value $H^{(0)}$ must be set based on different algorithms, among which the SHA-1, SHA-224, SHA-256, SHA-384, SHA-512, SHA-512/224, and SHA-512/256 algorithms use the initial Hash values (constant C) stored in the hardware.

However, SHA-512/ t requires a distinct initial hash value for each operation for a given value of t . Simply put, SHA-512/ t is the generic name for a t -bit hash function based on SHA-512 whose output is truncated to t bits. t is any positive integer without a leading zero such that $t < 512$, and t is not 384. The initial hash value for SHA-512/ t for a given value of t can be calculated by performing SHA-512 from hexadecimal representation of the string "SHA-512/ t ". It's not hard to observe that when determining the initial hash values for SHA-512/ t algorithms with different t , the only difference lies in the value of t .

Therefore, we have specially developed the following simplified method to calculate the initial hash value for SHA-512/ t :

1. **Generate t_string and t_length :** t_string is a 32-bit data that stores the input message of t . t_length is a 7-bit data that stores the length of the input message. The t_string and t_length are generated in methods described below, depending on the value of t :

- If $1 \leq t \leq 9$, then $t_length = 7'h48$ and t_string is padded in the following format:

$8'h30 + 8'ht_0$	$1'b1$	$23'b0$
------------------	--------	---------

where $t_0 = t$.

For example, if $t = 8$, then $t_0 = 8$ and $t_string = 32'h38800000$.

- If $10 \leq t \leq 99$, then $t_length = 7'h50$ and t_string is padded in the following format:

$8'h30 + 8'ht_1$	$8'h30 + 8'ht_0$	$1'b1$	$15'b0$
------------------	------------------	--------	---------

where, $t_0 = t \% 10$ and $t_1 = t / 10$.

For example, if $t = 56$, then $t_0 = 6$, $t_1 = 5$, and $t_string = 32'h35368000$.

- If $100 \leq t < 512$, then $t_length = 7'h58$ and t_string is padded in the following format:

$8'h30 + 8'ht_2$	$8'h30 + 8'ht_1$	$8'h30 + 8'ht_0$	$1'b1$	$7'b0$
------------------	------------------	------------------	--------	--------

where, $t_0 = t \% 10$, $t_1 = (t / 10) \% 10$, and $t_2 = t / 100$.

For example, if $t = 231$, then $t_0 = 1$, $t_1 = 3$, $t_2 = 2$, and $t_string = 32'h32333180$.

2. **Initialize relevant registers:** Initialize [SHA_T_STRING_REG](#) and [SHA_T_LENGTH_REG](#) with the generated t_string and t_length in the previous step.
3. **Obtain initial hash value:** Set the [SHA_MODE_REG](#) register to 7. Set the [SHA_START_REG](#) register to 1 to start the SHA accelerator. Then poll register [SHA_BUSY_REG](#) until the content of this register becomes 0, indicating the calculation of initial hash value is completed.

Please note that the initial value for SHA-512/ t can be also calculated according to the Section "5.3.6 SHA-512/ t " in [FIPS PUB 180-4 Spec](#), which is performing SHA-512 operation (with its initial hash value set to the result of 8-bitwise XOR operation of C and 0xa5) from the hexadecimal representation of the string "SHA-512/ t ".

29.4.2 Hash Operation

After the preprocessing, the ESP32-P4 SHA accelerator starts to hash a message M and generates message digest of different lengths, depending on different hash algorithms. As described above, the ESP32-P4 SHA accelerator supports two working modes, which are [Typical SHA](#) and [DMA-SHA](#). The operation process for the SHA accelerator under two working modes is described in the following subsections.

29.4.2.1 Typical SHA Mode Process

Usually, the SHA accelerator will process all blocks of a message and produce a message digest before starting the computation of the next message digest.

However, ESP32-P4 SHA also supports optional “interleaved” message digest calculation in Typical SHA mode, which means before SHA completes all blocks of the current message, users are given a chance to insert new computation of another message digest upon the completion of each individual block of the current message.

Specifically, users can read out the message digest from registers [SHA_H_n_REG](#) after completing part of a message digest calculation, and use the SHA accelerator for a different calculation. After the different calculation completes, users can restore the previous message digest to registers [SHA_H_n_REG](#), and resume the accelerator with the previously paused calculation.

Typical SHA Process (except for SHA-512/t)

1. Select a hash algorithm.
 - Configure the [SHA_MODE_REG](#) register based on Table [29.3-2](#).
2. Process the current message block.
 - Write the message block in registers [SHA_M_n_REG](#).
3. Start the SHA accelerator¹.
 - If this is the first time to execute this step, set the [SHA_START_REG](#) register to 1 to start the SHA accelerator. In this case, the accelerator uses the initial hash value stored in hardware for a given algorithm configured in Step 1 to start the calculation;
 - If this is not the first time to execute this step², set the [SHA_CONTINUE_REG](#) register to 1 to start the SHA accelerator. In this case, the accelerator uses the hash value stored in the [SHA_H_n_REG](#) register to start calculation.
4. Check the progress of the current message block.
 - Poll register [SHA_BUSY_REG](#) until the content of this register becomes 0, indicating the accelerator has completed the calculation for the current message block and now is in the “idle” status³.
5. Decide if you have more message blocks to process:
 - If yes, please go back to Step 2.
 - Otherwise, please continue.
6. Obtain the message digest.
 - Read the message digest from registers [SHA_H_n_REG](#).

Typical SHA Process (SHA-512/t)

1. Select a hash algorithm.
 - Configure the [SHA_MODE_REG](#) register to 7 for SHA-512/t.
2. Calculate the initial hash value.
 - (a) Calculate `t_string` and `t_length` and initialize [SHA_T_STRING_REG](#) and [SHA_T_LENGTH_REG](#) with the generated `t_string` and `t_length`. For details, please refer to Section [29.4.1.3](#).
 - (b) Set the [SHA_START_REG](#) register to 1 to start the SHA accelerator.
 - (c) Poll register [SHA_BUSY_REG](#) until the content of this register becomes 0, indicating the calculation of initial hash value is completed.
3. Process the current message block¹.
 - Write the message block in registers [SHA_M_n_REG](#).
4. Start the SHA accelerator
 - Set the [SHA_CONTINUE_REG](#) register to 1. In this case, the accelerator uses the hash value stored in the [SHA_H_n_REG](#) register to start calculation.
5. Check the progress of the calculation.
 - Poll register [SHA_BUSY_REG](#) until the content of this register becomes 0, indicating the accelerator has completed the calculation for the current message block and now is in the “idle” status³.
6. Decide if you have more message blocks to process:
 - If yes, please go back to Step 3.
 - Otherwise, please continue.
7. Obtain the message digest.
 - Read the message digest from registers [SHA_H_n_REG](#).

Note:

1. In this step, the software can also write the next message block (to be processed) in registers [SHA_M_n_REG](#), if any, while the hardware starts SHA calculation, to save time.
2. You are resuming the SHA accelerator with the previously paused calculation.
3. Here you can decide if you want to insert other calculations. If yes, please go to the [process for interleaved calculations](#) for details.

As mentioned above, ESP32-P4 SHA accelerator supports “interleaving” calculation under the **Typical SHA working mode**.

The process to implement interleaved calculation is described below.

1. Prepare to hand the SHA accelerator over for an interleaved calculation by storing the following data of the previous calculation.
 - The selected hash algorithm configured in the [SHA_MODE_REG](#) register.
 - The message digest stored in registers [SHA_H_n_REG](#).

2. Perform the interleaved calculation. For the detailed process of the interleaved calculation, please refer to [Typical SHA process](#) or [DMA-SHA process](#), depending on the working mode of your interleaved calculation.
3. Prepare to hand the SHA accelerator back to the previously paused calculation by restoring the following data of the previous calculation.
 - Write the previously stored hash algorithm back to register [SHA_MODE_REG](#).
 - Write the previously stored message digest back to registers [SHA_H_n_REG](#).
4. Write the next message block from the previous paused calculation in registers [SHA_M_n_REG](#), and set the [SHA_CONTINUE_REG](#) register to 1 to restart the SHA accelerator with the previously paused calculation.

29.4.2.2 DMA-SHA Mode Process

ESP32-P4 SHA accelerator does not support “interleaving” message digest calculation at the level of individual message blocks when using DMA, which means you cannot insert new calculation before a complete DMA-SHA process (of one or more message blocks) completes. In this case, users who need interleaved operation are recommended to divide the message blocks and perform several DMA-SHA calculations, instead of trying to compute all the messages in one go.

Single DMA-SHA calculation supports up to 63 message blocks.

In contrast to the Typical SHA working mode, when the SHA accelerator is working under the DMA-SHA mode, all data read are completed via GDMA-AXI. Therefore, users are required to configure the GDMA-AXI controller following the description in Chapter 4 [GDMA Controller \(GDMA-AHB, GDMA-AXI\)](#).

DMA-SHA process (except SHA-512/t)

1. Select a hash algorithm.
 - Select a hash algorithm by configuring the [SHA_MODE_REG](#) register. For details, please refer to Table 29.3-2.
2. Configure the [SHA_INT_ENA_REG](#) register to enable or disable interrupt (Set 1 to enable).
3. Configure the number of message blocks.
 - Write the number of message blocks M to the [SHA_DMA_BLOCK_NUM_REG](#) register.
4. Start the DMA-SHA calculation.
 - If the current DMA-SHA calculation follows a previous calculation, firstly write the message digest from the previous calculation to registers [SHA_H_n_REG](#), then write 1 to register [SHA_DMA_CONTINUE_REG](#) to start SHA accelerator;
 - Otherwise, write 1 to register [SHA_DMA_START_REG](#) to start the accelerator.
5. Wait till the completion of the DMA-SHA calculation, which happens when:
 - The content of [SHA_BUSY_REG](#) register becomes 0, or
 - An SHA interrupt occurs. In this case, please clear interrupt by writing 1 to the [SHA_INT_CLEAR_REG](#) register.

6. Obtain the message digest:

- Read the message digest from registers [SHA_H_*n*_REG](#).

DMA-SHA process for SHA-512/*t*

1. Select a hash algorithm.

- Select SHA-512/*t* algorithm by configuring the [SHA_MODE_REG](#) register to 7.

2. Configure the [SHA_INT_ENA_REG](#) register to enable or disable interrupt (Set 1 to enable).

3. Calculate the initial hash value.

- (a) Calculate *t_string* and *t_length* and initialize [SHA_T_STRING_REG](#) and [SHA_T_LENGTH_REG](#) with the generated *t_string* and *t_length*. For details, please refer to Section [29.4.1.3](#).
- (b) Set the [SHA_START_REG](#) register to 1 to start the SHA accelerator.
- (c) Poll register [SHA_BUSY_REG](#) until the content of this register becomes 0, indicating the calculation of initial hash value is completed.

4. Configure the number of message blocks.

- Write the number of message blocks *M* to the [SHA_DMA_BLOCK_NUM_REG](#) register.

5. Start the DMA-SHA calculation.

- Write 1 to register [SHA_DMA_CONTINUE_REG](#) to start the accelerator.

6. Wait till the completion of the DMA-SHA calculation, which happens when:

- The content of [SHA_BUSY_REG](#) register becomes 0, or
- An SHA interrupt occurs. In this case, please clear interrupt by writing 1 to the [SHA_INT_CLEAR_REG](#) register.

7. Obtain the message digest:

- Read the message digest from registers [SHA_H_*n*_REG](#).

29.4.3 Message Digest

After the hash task completes, the SHA accelerator writes the message digest from the task to registers [SHA_H_*n*_REG](#) (*n*: 0 ~ 15). The lengths of the generated message digest are different depending on different hash algorithms. For details, see Table [29.4-4](#) below:

Table 29.4-4. The Storage and Length of Message Digest from Different Algorithms

Hash Algorithm	Length of Message Digest (in bits)	Storage ¹
SHA-1	160	SHA_H_O_REG ~ SHA_H_4_REG
SHA-224	224	SHA_H_O_REG ~ SHA_H_6_REG
SHA-256	256	SHA_H_O_REG ~ SHA_H_7_REG
SHA-384	384	SHA_H_O_REG ~ SHA_H_11_REG
SHA-512	512	SHA_H_O_REG ~ SHA_H_15_REG
SHA-512/224	224	SHA_H_O_REG ~ SHA_H_6_REG
SHA-512/256	256	SHA_H_O_REG ~ SHA_H_7_REG
SHA-512/ t^2	t	SHA_H_O_REG ~ SHA_H_x_REG

¹ The message digest is stored in registers from most significant bits to the least significant bits, with the first word stored in register [SHA_H_O_REG](#) and the second word stored in register [SHA_H_1_REG](#)... For details, please see subsection [29.4.1.2](#).

² The registers used for SHA-512/ t algorithm depend on the value of t . $x+1$ indicates the number of 32-bit registers used to store t bits of message digest, so that $x = \text{roundup}(t/32)-1$. For example:

- When $t = 8$, then $x = 0$, indicating that the 8-bit long message digest is stored in the most significant 8 bits of register [SHA_H_O_REG](#);
- When $t = 32$, then $x = 0$, indicating that the 32-bit long message digest is stored in register [SHA_H_O_REG](#);
- When $t = 132$, then $x = 4$, indicating that the 132-bit long message digest is stored in registers [SHA_H_O_REG](#), [SHA_H_1_REG](#), [SHA_H_2_REG](#), [SHA_H_3_REG](#), and [SHA_H_4_REG](#).

29.5 Interrupt

ESP32-P4's SHA accelerator can generate the following interrupt signal(s) that will be sent to the [Interrupt Matrix](#).

- SHA_INTR

There are several internal interrupt sources from SHA that can generate the above interrupt signal(s). The interrupt sources from SHA are listed with their trigger conditions and the resulted interrupt signal(s) in [Table 29.5-1](#).

Table 29.5-1. SHA's Internal Interrupt Sources

Internal Interrupt Source	Trigger Condition	Interrupt Signal
SHA_CALC_DONE_INT	Completion of an message digest calculation in DMA-SHA mode	SHA_INTR

Note:

- For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to [Chapter 12](#)

Interrupt Matrix > Section 12.2 *Interrupt Terminology in ESP32-P4*.

- Different from the standard interrupt register group, the interrupt register group of SHA only contains the INT_ENA ([SHA_INT_ENA_REG](#)) and INT_CLR ([SHA_INT_CLEAR_REG](#)) fields, and does not support users to read the INT_RAW and INT_ST fields.

29.6 Register Summary

The addresses in this section are relative to SHA accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Control/Configuration Registers			
SHA_MODE_REG	Configures SHA algorithm	0x0000	R/W
SHA_T_STRING_REG	Initial Hash Value calculation factor t_string (for SHA-512/t)	0x0004	R/W
SHA_T_LENGTH_REG	Initial Hash Value calculation factor t_length (for SHA-512/t)	0x0008	R/W
SHA_CONTINUE_REG	Continues SHA operation (only effective in Typical SHA mode)	0x0014	WO
SHA_DMA_START_REG	Starts the SHA accelerator for DMA-SHA operation	0x001C	WO
SHA_START_REG	Starts the SHA accelerator for Typical SHA operation	0x0010	WO
SHA_DMA_CONTINUE_REG	Continues SHA operation (only effective in DMA-SHA mode)	0x0020	WO
SHA_DMA_BLOCK_NUM_REG	Block number register (only effective in DMA-SHA mode)	0x000C	R/W
Status Registers			
SHA_BUSY_REG	Represents if SHA Accelerator is busy or not	0x0018	RO
Interrupt Registers			
SHA_INT_CLEAR_REG	DMA-SHA interrupt clear register	0x0024	WO
SHA_INT_ENA_REG	DMA-SHA interrupt enable register	0x0028	R/W
Data Registers			
SHA_H_0_REG	Hash value	0x0040	R/W
SHA_H_1_REG	Hash value	0x0044	R/W
SHA_H_2_REG	Hash value	0x0048	R/W
SHA_H_3_REG	Hash value	0x004C	R/W
SHA_H_4_REG	Hash value	0x0050	R/W
SHA_H_5_REG	Hash value	0x0054	R/W
SHA_H_6_REG	Hash value	0x0058	R/W
SHA_H_7_REG	Hash value	0x005C	R/W
SHA_H_8_REG	Hash value	0x0060	R/W
SHA_H_9_REG	Hash value	0x0064	R/W
SHA_H_10_REG	Hash value	0x0068	R/W
SHA_H_11_REG	Hash value	0x006C	R/W
SHA_H_12_REG	Hash value	0x0070	R/W
SHA_H_13_REG	Hash value	0x0074	R/W
SHA_H_14_REG	Hash value	0x0078	R/W
SHA_H_15_REG	Hash value	0x007C	R/W
SHA_M_0_REG	Message	0x0080	R/W
SHA_M_1_REG	Message	0x0084	R/W
SHA_M_2_REG	Message	0x0088	R/W
SHA_M_3_REG	Message	0x008C	R/W
SHA_M_4_REG	Message	0x0090	R/W
SHA_M_5_REG	Message	0x0094	R/W

Name	Description	Address	Access
SHA_M_6_REG	Message	0x0098	R/W
SHA_M_7_REG	Message	0x009C	R/W
SHA_M_8_REG	Message	0x00A0	R/W
SHA_M_9_REG	Message	0x00A4	R/W
SHA_M_10_REG	Message	0x00A8	R/W
SHA_M_11_REG	Message	0x00AC	R/W
SHA_M_12_REG	Message	0x00B0	R/W
SHA_M_13_REG	Message	0x00B4	R/W
SHA_M_14_REG	Message	0x00B8	R/W
SHA_M_15_REG	Message	0x00BC	R/W
SHA_M_16_REG	Message	0x00C0	R/W
SHA_M_17_REG	Message	0x00C4	R/W
SHA_M_18_REG	Message	0x00C8	R/W
SHA_M_19_REG	Message	0x00CC	R/W
SHA_M_20_REG	Message	0x00D0	R/W
SHA_M_21_REG	Message	0x00D4	R/W
SHA_M_22_REG	Message	0x00D8	R/W
SHA_M_23_REG	Message	0x00DC	R/W
SHA_M_24_REG	Message	0x00E0	R/W
SHA_M_25_REG	Message	0x00E4	R/W
SHA_M_26_REG	Message	0x00E8	R/W
SHA_M_27_REG	Message	0x00EC	R/W
SHA_M_28_REG	Message	0x00F0	R/W
SHA_M_29_REG	Message	0x00F4	R/W
SHA_M_30_REG	Message	0x00F8	R/W
SHA_M_31_REG	Message	0x00FC	R/W
Version Register			
SHA_DATE_REG	Version control register	0x002C	R/W

29.7 Registers

The addresses in this section are relative to SHA accelerator base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 29.1. SHA_MODE_REG (0x0000)

(reserved)																								SHA_MODE		
31																								3	2	0
0 0																								0x0		Reset

SHA_MODE Configures the SHA algorithm.

- 0: SHA-1
- 1: SHA-224
- 2: SHA-256
- 3: SHA-384
- 4: SHA-512
- 5: SHA-512/224
- 6: SHA-512/256
- 7: SHA-512/t
- (R/W)

Register 29.2. SHA_T_STRING_REG (0x0004)

SHA_T_STRING																															
31																															0
0x000000																															
Reset																															

SHA_T_STRING Configures t_string for calculating the initial Hash value for SHA-512/t. (R/W)

Register 29.3. SHA_T_LENGTH_REG (0x0008)

(reserved)																						SHA_T_LENGTH					
31																						7	6	0			
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x0				Reset

SHA_T_LENGTH Configures t_length for calculating the initial Hash value for SHA-512/t. (R/W)

Register 29.4. SHA_DMA_BLOCK_NUM_REG (0x000C)

(reserved)																SHA_DMA_BLOCK_NUM																
31															6	5											0					
0 0																0x0																Reset

Reset

SHA_DMA_BLOCK_NUM Configures the DMA-SHA block number. (R/W)

Register 29.5. SHA_START_REG (0x0010)

(reserved)																																SHA_START																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_START Write 1 to start Typical SHA calculation. (WO)

Register 29.6. SHA_CONTINUE_REG (0x0014)

(reserved)																																SHA_CONTINUE																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_CONTINUE Write 1 to continue Typical SHA calculation. (WO)

Register 29.7. SHA_BUSY_REG (0x0018)

(reserved)																															SHA_BUSY_STATE																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																															
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_BUSY_STATE Represents the states of SHA accelerator.

0: idle

1: busy

(RO)

Register 29.8. SHA_DMA_START_REG (0x001C)

(reserved)																														SHA_DMA_START	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_DMA_START Write 1 to start DMA-SHA calculation. (WO)

Register 29.9. SHA_DMA_CONTINUE_REG (0x0020)

(reserved)																														SHA_DMA_CONTINUE	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_DMA_CONTINUE Write 1 to continue DMA-SHA calculation. (WO)

Register 29.10. SHA_INT_CLEAR_REG (0x0024)

(reserved)																														SHA_CLEAR_INTERRUPT	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_CLEAR_INTERRUPT Write 1 to clear DMA-SHA interrupt. (WO)

Register 29.11. SHA_INT_ENA_REG (0x0028)

(reserved)																														SHA_INTERRUPT_ENA	
31																														1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

SHA_INTERRUPT_ENA Write 1 to enable DMA-SHA interrupt. (R/W)

(reserved)

SHA_DATE

Reset

SHA_DATE Version control register. (R/W)

Register 29.13. SHA_H_n_REG (*n*: 0-15) (0x0040+4**n*)

SHA_H_n

Reset

SHA_H_*n* Represents the *n*th 32-bit piece of the Hash value. (R/W)

Register 29.14. SHA_M_n_REG (n: 0-31) (0x0080+4*n)

SHA_M_n

Reset

SHA_M_n Represents the *n*th 32-bit piece of the message. (R/W)